

CPPDescent

74ba700 (with uncommitted changes)

Generated by Doxygen 1.9.6

1 CPP-Descent	1
1.1 Dependencies	1
1.1.1 Minor dependencies	1
1.2 Contributors	1
1.3 Bibliography	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 cppdescent Class Reference	7
4.1.1 Detailed Description	7
4.1.2 Member Function Documentation	7
4.1.2.1 hello()	7
4.2 List Class Reference	8
4.2.1 Detailed Description	8
4.2.2 Constructor & Destructor Documentation	8
4.2.2.1 List()	8
4.2.2.2 ~List()	9
4.2.3 Member Function Documentation	9
4.2.3.1 find()	9
4.2.3.2 findNode()	9
4.2.3.3 getHead()	10
4.2.3.4 getSize()	10
4.2.3.5 getTail()	10
4.2.3.6 insertNext()	10
4.2.3.7 next()	11
4.2.3.8 nodeValue()	11
4.2.3.9 removeNext()	11
4.2.3.10 setDestroyValue()	12
4.3 ListNode Class Reference	12
4.3.1 Detailed Description	13
4.3.2 Constructor & Destructor Documentation	13
4.3.2.1 ListNode() [1/2]	13
4.3.2.2 ListNode() [2/2]	13
4.3.3 Member Function Documentation	13
4.3.3.1 getNext()	13
4.3.3.2 getValue()	14
4.3.3.3 setNext()	14
5 File Documentation	15

5.1 common.hpp	15
5.2 cppdescent.hpp	15
5.3 linkedList.hpp File Reference	15
5.3.1 Detailed Description	16
5.4 linkedList.hpp	16
5.5 cppdescent.cpp File Reference	17
Index	19

Chapter 1

CPP-Descent

Part of the "Software Development for Computing Systems" course of the Department of Informatics and Telecommunications, UoA for the first semester of the academic year 2023-2024.

A library that implements the KNN-Graph creation and search algorithms described in [1], [2].

1.1 Dependencies

For local development, you will need `Make` and `CMake`. To install them on an apt-based linux distro, run the following commands:

```
$ sudo apt update && sudo apt upgrade
$ sudo apt install make
$ sudo apt install cmake
```

1.1.1 Minor dependencies

- The project uses `Google Test` for its testing needs. Gtest is used as a git submodule of this repo, that is downloaded by CMake the first time you clone. That means that nothing extra needs to be installed to run the tests.
- If you would like to generate the documentation, you will need `doxygen` installed. ## Code Structure

In the `wiki`, you can find documentation generated by `Doxygen`. It is also available in `pdf form`.

1.2 Contributors

- `Konstantinos Chousos` (1115202000215)
- `Anastasios-Fedon Seitanidis` (1115202000179)

1.3 Bibliography

[1] W. Dong, C. Moses, and K. Li, "Efficient k-nearest neighbor graph construction for generic similarity measures," in *Proceedings of the 20th international conference on World wide web*, in WWW '11. New York, NY, USA: Association for Computing Machinery, Mar. 2011, pp. 577–586. doi: [10.1145/1963405.1963487](https://doi.org/10.1145/1963405.1963487).

[2] "How PyNNDescent works — pynndescent 0.5.0 documentation." Accessed: Oct. 10, 2023. [Online]. Available: https://pynndescent.readthedocs.io/en/latest/how_pynndescent_works.html

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

cppdescent	A sample function for the cppdescent class, used to test the correctness of the CMake files . . .	7
List	ADT List	8
ListNode	Node of a list	12

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

common.hpp	15
cppdescent.hpp	15
linkedList.hpp	
Header file for an ADT List implementation	15
cppdescent.cpp	
Implementation of cppdescent library	17

Chapter 4

Class Documentation

4.1 cppdescent Class Reference

A sample function for the cppdescent class, used to test the correctness of the CMake files.

```
#include <cppdescent.hpp>
```

Public Member Functions

- int [hello](#) (void)
Prints "Hello World".

4.1.1 Detailed Description

A sample function for the cppdescent class, used to test the correctness of the CMake files.

4.1.2 Member Function Documentation

4.1.2.1 hello()

```
int cppdescent::hello (  
    void )
```

Prints "Hello World".

Returns

int exit code

The documentation for this class was generated from the following files:

- [cppdescent.hpp](#)
- [cppdescent.cpp](#)

4.2 List Class Reference

ADT [List](#).

```
#include <LinkedList.hpp>
```

Public Member Functions

- [List](#) (DestroyFunc destroyValue=nullptr)
Construct a new [List](#) object.
- [~List](#) ()
Destroy the [List](#) object.
- void [insertNext](#) ([ListNode](#) node, Pointer value)
Adds a node after the given node.
- void [removeNext](#) ([ListNode](#) node)
Removes the node after the given node.
- Pointer [find](#) (Pointer value, CompareFunc compare)
Finds the first value equal to the value parameter.
- DestroyFunc [setDestroyValue](#) (DestroyFunc value)
Set the Destroy Value object to a new value.
- int [getSize](#) ()
Get the size of the [List](#).
- int [increaseSize](#) ()
- int [decreaseSize](#) ()
- [ListNode](#) * [getHead](#) ()
Getter for the head of the [List](#).
- [ListNode](#) * [getTail](#) ()
Getter for the tail of the [List](#).
- [ListNode](#) [next](#) ([ListNode](#) node)
Returns the node after the given one.
- Pointer [nodeValue](#) ([ListNode](#) node)
Returns the value of the node.
- [ListNode](#) [findNode](#) (Pointer value, CompareFunc compare)
Finds the first node that has value equal to the value parameter.

4.2.1 Detailed Description

ADT [List](#).

An Abstract Data Type [List](#) with linear access to data, and addition and removal to/from arbitrary positions.

4.2.2 Constructor & Destructor Documentation

4.2.2.1 List()

```
List::List (
    DestroyFunc destroyValue = nullptr )
```

Construct a new [List](#) object.

Parameters

<i>destroyValue</i>	If <i>destroyValue</i> != nullptr, the <i>destroyValue</i> function will be called each time a node is removed.
---------------------	---

4.2.2.2 ~List()

```
List::~~List ( )
```

Destroy the [List](#) object.

4.2.3 Member Function Documentation**4.2.3.1 find()**

```
Pointer List::find (
    Pointer value,
    CompareFunc compare )
```

Finds the first value equal to the value parameter.

Parameters

<i>value</i>	The value based on which the search is done.
<i>compare</i>	A pointer to the function that will compare the node values.

Returns

Pointer A pointer to the resulting node.

4.2.3.2 findNode()

```
ListNode List::findNode (
    Pointer value,
    CompareFunc compare )
```

Finds the first node that has value equal to the value parameter.

Parameters

<i>value</i>	The value to search.
<i>compare</i>	The compare function to use.

Returns

[ListNode](#) A pointer to the resulting node.

4.2.3.3 getHead()

```
ListNode * List::getHead ( )
```

Getter for the head of the [List](#).

Returns

ListNode* A pointer to the head.

4.2.3.4 getSize()

```
int List::getSize ( )
```

Get the size of the [List](#).

The size is updated upon entry/removal of nodes, so the retrieval of the size is O(1).

Returns

int

4.2.3.5 getTail()

```
ListNode * List::getTail ( )
```

Getter for the tail of the [List](#).

Returns

ListNode* A pointer to the tail.

4.2.3.6 insertNext()

```
void List::insertNext (
    ListNode node,
    Pointer value )
```

Adds a node *after* the given node.

Parameters

<i>node</i>	The node that the new node will be placed after.
<i>value</i>	The value of the new node.

4.2.3.7 next()

```
ListNode List::next (
    ListNode node )
```

Returns the node after the given one.

Parameters

<i>node</i>	
-------------	--

Returns

[ListNode](#) A pointer to the next node.

4.2.3.8 nodeValue()

```
Pointer List::nodeValue (
    ListNode node )
```

Returns the value of the node.

Parameters

<i>node</i>	
-------------	--

Returns

Pointer A generic pointer to the value.

4.2.3.9 removeNext()

```
void List::removeNext (
    ListNode node )
```

Removes the node *after* the given node.

Parameters

<i>node</i>	The node before the node to be removed.
-------------	---

4.2.3.10 setDestroyValue()

```
DestroyFunc List::setDestroyValue (  
    DestroyFunc value )
```

Set the Destroy Value object to a new value.

Parameters

<i>value</i>	A pointer to the new destroy funciton.
--------------	--

Returns

DestroyFunc A pointer to the old destroy function.

The documentation for this class was generated from the following files:

- [linkedList.hpp](#)
- [linkedList.cpp](#)

4.3 ListNode Class Reference

Node of a list.

```
#include <linkedList.hpp>
```

Public Member Functions

- [ListNode](#) ()
Construct a new [List](#) Node object that is empty.
- [ListNode](#) (Pointer value)
Construct a new [List](#) Node object with value 'value'.
- void [setNext](#) ([ListNode](#) *next)
Setter for the next node.
- [ListNode](#) * [getNext](#) ()
Getter for the next node.
- Pointer [getValue](#) ()
Getter for the value of the node.

4.3.1 Detailed Description

Node of a list.

A list node is represented by its value. It also holds a pointer to the next node in the list.

4.3.2 Constructor & Destructor Documentation

4.3.2.1 ListNode() [1/2]

```
ListNode::ListNode ( )
```

Construct a new [List](#) Node object that is empty.

4.3.2.2 ListNode() [2/2]

```
ListNode::ListNode (
    Pointer value ) [inline]
```

Construct a new [List](#) Node object with value 'value'.

Parameters

<i>value</i>	a generic pointer to the value of the node.
--------------	---

4.3.3 Member Function Documentation

4.3.3.1 getNext()

```
ListNode * ListNode::getNext ( )
```

Getter for the next node.

Returns

ListNode* The pointer to the next node.

4.3.3.2 `getValue()`

```
Pointer ListNode::getValue ( )
```

Getter for the value of the node.

Returns

Pointer A generic pointer to the value.

4.3.3.3 `setNext()`

```
void ListNode::setNext (
    ListNode * next )
```

Setter for the next node.

Parameters

<i>next</i>	The pointer to the next node.
-------------	-------------------------------

The documentation for this class was generated from the following files:

- [linkedList.hpp](#)
- [listNode.cpp](#)

Chapter 5

File Documentation

5.1 common.hpp

```
00001 #pragma once
00002
00003 typedef void* Pointer;
00004
00005 typedef int (*CompareFunc)(Pointer a, Pointer b);
00006
00007 typedef void (*DestroyFunc)(Pointer value);
```

5.2 cppdescent.hpp

```
00001
00006 class cppdescent {
00007 public:
00013     int hello(void);
00014 };
```

5.3 linkedList.hpp File Reference

Header file for an ADT [List](#) implementation.

```
#include "common.hpp"
```

Include dependency graph for linkedList.hpp:

Classes

- class [ListNode](#)

Node of a list.

- class [List](#)

ADT [List](#).

Macros

- #define **LIST_BOF** ([ListNode](#))0
Virtual node before the first node.
- #define **LIST_EOF** ([ListNode](#))0
Virtual node after the last node.

5.3.1 Detailed Description

Header file for an ADT [List](#) implementation.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-28

Copyright

Copyright (c) 2023

5.4 linkedList.hpp

[Go to the documentation of this file.](#)

```
00001
00011 #pragma once
00012 #include "common.hpp"
00013
00014 #define LIST_BOF (ListNode)0
00015 #define LIST_EOF (ListNode)0
00016
00024 class ListNode {
00025 public:
00030     ListNode();
00036     ListNode(Pointer value) : value(value){};
00037     // ~ListNode();
00043     void setNext(ListNode* next);
00049     ListNode* getNext();
00055     Pointer getValue();
00056
00057 private:
00058     ListNode* next;
00059     Pointer value;
00060 };
00061
00069 class List {
00070 public:
00077     List(DestroyFunc destroyValue = nullptr);
00082     ~List();
00089     void insertNext(ListNode node, Pointer value);
00095     void removeNext(ListNode node);
00103     Pointer find(Pointer value, CompareFunc compare);
00110     DestroyFunc setDestroyValue(DestroyFunc value);
00119     int getSize();
00120     int increaseSize();
00121     int decreaseSize();
00127     ListNode* getHead();
00133     ListNode* getTail();
00140     ListNode next(ListNode node);
00147     Pointer nodeValue(ListNode node);
00155     ListNode findNode(Pointer value, CompareFunc compare);
00156
00157 private:
00158     ListNode* dummy;
00159     ListNode* head;
00160     ListNode* tail;
00161     int size;
00162     DestroyFunc destroyValue;
00163 };
```

5.5 cppdescent.cpp File Reference

Implementation of cppdescent library.

```
#include "cppdescent/cppdescent.hpp"
```

```
#include <iostream>
```

Include dependency graph for cppdescent.cpp:

Index

- ~List
 - List, [9](#)
- common.hpp, [15](#)
- cppdescent, [7](#)
 - hello, [7](#)
- cppdescent.cpp, [17](#)
- cppdescent.hpp, [15](#)
- find
 - List, [9](#)
- findNode
 - List, [9](#)
- getHead
 - List, [10](#)
- getNext
 - ListNode, [13](#)
- getSize
 - List, [10](#)
- getTail
 - List, [10](#)
- getValue
 - ListNode, [13](#)
- hello
 - cppdescent, [7](#)
- insertNext
 - List, [10](#)
- linkedList.hpp, [15](#), [16](#)
- List, [8](#)
 - ~List, [9](#)
 - find, [9](#)
 - findNode, [9](#)
 - getHead, [10](#)
 - getSize, [10](#)
 - getTail, [10](#)
 - insertNext, [10](#)
 - List, [8](#)
 - next, [11](#)
 - nodeValue, [11](#)
 - removeNext, [11](#)
 - setDestroyValue, [12](#)
- ListNode, [12](#)
 - getNext, [13](#)
 - getValue, [13](#)
 - ListNode, [13](#)
 - setNext, [14](#)
- next
 - List, [11](#)
- nodeValue
 - List, [11](#)
- removeNext
 - List, [11](#)
- setDestroyValue
 - List, [12](#)
- setNext
 - ListNode, [14](#)